

Kavach: Deterministic-Primary Detection for LLM Agent Tool Calls

A Rearchitecture, a Real-Trajectory Validation, and Negative Results

Anonymous Author(s)

Abstract

LLM agents increasingly act in the world through tool calls, but deployed guardrails overwhelmingly evaluate an agent’s language rather than the resolved arguments of the action it is about to take. We present Kavach, a runtime monitor that intercepts every tool call at the moment its arguments are resolved, via a parliament of four domain-specialized ministers — VAULT (credentials), EXECUTOR (dangerous code), CHANNEL (exfiltration), and NAVIGATOR (unauthorized action) — whose verdicts a deterministic speaker combines into Block, Escalate, or Allow. This paper reports four contributions. First, a methodology finding: across independent lines of evidence, a deterministic-primary architecture — structural parsing over resolved arguments, embedding similarity demoted to escalate-only triage — beats the cosine-primary design it replaced when the attack signal is structural, on both recall and false-positive rate, moving three of four detectors. Second, an evaluation across InjecAgent, AgentDojo, and a composite corpus assembled for the two detectors those benchmarks barely exercise (in-sample, and labeled as such, with an independent check reported alongside it), reporting benign false-positive and Escalate rates jointly with recall throughout rather than recall alone. Third, a large-scale validation against real recorded GPT-4o attack trajectories, including a benign-trajectory control under the identical firing criterion, so the headline detection rate has an honest baseline to be read against. Fourth, negative results reported in full: failed authorization heuristics, an abandoned deterministic swap, independent generalization checks against both an author-written novel vocabulary and an externally published attack corpus, and a benign-side gap found and fixed by direct measurement during this work. Designed to run offline on CPU-only hardware, no GPU required.

1 Introduction

A tool call to `send_money` with an attacker-substituted recipient IBAN, or to a shell one-liner that quietly harvests credentials, is often indistinguishable from its benign counterpart at the level of language — the surrounding prompt, the model’s chain-of-thought, the plan it wrote down all read the same either way. The two calls only come apart once their concrete, resolved arguments are inspected: the actual IBAN, the actual command string. Most deployed guardrails never look there. They evaluate what the agent *said* it would do, not what the action it is about to take will actually resolve to. Kavach closes that gap: a runtime monitor that intercepts every tool call at the moment its arguments are resolved and

scores the resulting action — not the prompt, not the plan, the resolved call itself — before any side effect occurs.

This paper is substantially a report on a design decision and the evidence that produced it: we built Kavach’s first version as a semantic, embedding-based detector — a parliament of domain-specialized ministers, each retrieving against a curated corpus of attack-pattern descriptions — and over the course of stress-testing it, accumulated four independent lines of evidence that this design was the wrong primary mechanism for three of its four detection domains, while remaining right for the fourth. We report both the evidence and the resulting rearchitecture, including attempts that did not work, since a paper reporting only the final configuration understates how the field should update: not every detection problem shaped by resolved arguments is best solved by comparing embeddings.

Contributions. First: deterministic-primary beats cosine-primary when the attack signal is structural (§3).

Four independent lines of evidence make this case: a lexical-inflation bug, a dense fine-tune that traded one failure mode for a worse one, a corpus-coverage gap that generalized past its original motivating case, and a proposed fifth minister formally falsified before we built it, since paired legitimate/malicious tool calls differing only in argument content show near-zero cosine similarity deltas (0.003–0.024). Together, these show that credential-path shapes, network-fetch-to-disk patterns, and destination-value provenance are answered more reliably by deterministic parsing over an action’s resolved arguments than by any refinement of embedding similarity over its semantic content. We moved three of Kavach’s four ministers (VAULT, EXECUTOR, CHANNEL) to deterministic-primary detection on this evidence, keeping cosine similarity only as an escalate-only triage layer over the deterministic residual.

Second, a working system with real evaluation numbers (§3, §4).

InjecAgent and AgentDojo, the standard benchmarks we also evaluate against below, almost entirely miss VAULT and EXECUTOR — both stress CHANNEL specifically, a gap found live while validating this rearchitecture. We assembled a composite evaluation corpus (§4.2) for VAULT and EXECUTOR from real, externally-validated attack data (AtomicRedTeam, GTF0Bins) rather than authoring new synthetic attacks risking unconscious shaping toward the evaluating system; CHANNEL and NAVIGATOR are evaluated separately on InjecAgent and AgentDojo below. Full numbers and baselines are in §4: VAULT 74.4%, EXECUTOR 90.1%, zero new benign false positives, 544/544 on InjecAgent’s data-stealing corpus via CHANNEL’s taint

mechanism (direct-harm remains open), and 5/5 structurally applicable AgentDojo attacks correctly escalated with zero false blocks on 22 real benign sessions.

Third, a large-scale real-trajectory validation (§4.5). Beyond benchmark replay, we validated Kavach against all 629 real recorded GPT-4o attack trajectories published by AgentDojo — actual model-generated tool-call sequences with real tool outputs, not a benchmark’s minimal answer key. Kavach fires on 467/629 (74.2%); tracing every apparent miss to its actual tool-call content, most reflect the model never taking any consequential action at all (an artifact of AgentDojo’s own success-labeling convention, not a detection gap), leaving 19 genuine misses, each categorized precisely by mechanism (§4.5).

Fourth, honestly reported negative results (§6). NAVIGATOR, the one minister not moved to deterministic primary detection, is reported in full: an abandoned deterministic swap, and three independently designed authorization heuristics that each failed a frozen discriminability criterion, for three distinct reasons. Independent generalization checks against a fictional agent vocabulary and an externally-published attack corpus show precisely which mechanisms transfer — EXECUTOR cleanly, VAULT mostly, CHANNEL’s escalation path but not its hard-block path, NAVIGATOR not at all (§6).

Scope and non-claims. We do not claim Kavach prevents every class of agent-side attack: its deterministic rules are authored against published taxonomies and known attack corpora, and a genuinely novel technique outside both may not be caught, characterized precisely by the generalization audit (§6) rather than left as an assumption. NAVIGATOR’s authorization question — was this action derivable from what the user asked for — remains open past a validated taxonomy classifier; we report the attempts and failure modes, not a completed mechanism. Nor do we propose Kavach as a replacement for input-time or output-time guardrails: it enforces at a different lifecycle boundary, and a comprehensive deployment runs more than one.

Reproducibility. The parliament service, deterministic pre-filters and test suites, evaluation harness, and replay scripts behind every number in §4 run entirely offline on CPU-only hardware, no GPU or paid API access required.

2 Background

2.1 The agent infrastructure attack surface

Modern LLM agents are composite systems: a language model provides the reasoning surface, while the harness provides tool dispatch, context management, multi-turn state, and persistence. In production deployments the model is typically an external API call, while tool dispatch, context management, multi-turn state, and persistence are all local harness code — a structural asymmetry with a direct security consequence: the harness, not the model itself, is where most exploitable surface area lives.

Agent harnesses expose several classes of latent vulnerabilities. *Configuration-as-code* attacks exploit declarative configuration the harness loads and executes unsanitized (CVE-2026-21852 on Claude Code’s `.claude/settings.json`; CVE-2026-21858 on n8n). *Serialization-injection* attacks exploit serialization/deserialization of tool responses and agent state, enabling arbitrary class instantiation in trusted namespaces (CVE-2025-68664 on LangChain Core, CVSS 9.3). *Hook-bypass* attacks exploit incorrectly wired security hook points (OpenClaw issues #5513, #5943: `before.tool.call` documented but never invoked), and *CSRF/session* attacks exploit the harness’s web surface (CVE-2025-34291 on LangFlow, actively exploited in the wild as of January 2026 [7]).

These vulnerability classes correspond directly to Kavach’s minister structure: EXECUTOR detects code-execution-class attacks (CVE-2025-59536, CVE-2025-34291, CVE-2025-68664), VAULT credential-theft-class attacks (CVE-2026-21852, CVE-2025-68664), CHANNEL exfiltration-class attacks (CVE-2026-21852 via URL redirection, CVE-2026-21858), and NAVIGATOR trajectory-manipulation attacks spanning multiple tool calls. Section 3 gives the MITRE ATT&CK and OWASP Agentic 2026 taxonomic grounding for each minister.

2.2 Existing guardrails and their enforcement points

The guardrail literature can be organized along two axes: (1) *where* in the agent lifecycle the guardrail enforces, and (2) *what mechanism* it uses to detect violations.

Input-time guardrails. (PromptGuard 2 [5], Azure Prompt Shields) operate on the user’s message before the agent processes it, and cannot detect attacks arriving via tool outputs, the dominant vector in prompt-injection benchmarks [21].

Plan-time / chain-of-thought guardrails. (AlignmentCheck [5], DRIFT [15]) audit the agent’s reasoning trace before it commits to an action — powerful but expensive (full LLM inference), and blind to a tool call’s *resolved arguments*, seeing only declarative intent.

Tool-call-time guardrails. operate at the tool-call boundary — Kavach’s enforcement point. CodeShield [5] applies Semgrep to code-returning outputs only; ShieldAgent [4] retrieves rule circuits per trajectory; Task Shield [12] checks task alignment. None combines deterministic parsing of an action’s resolved arguments with a semantic corpus as a secondary triage layer at the tool-call level — the combination §3 argues for, after finding the semantic corpus alone was the wrong primary mechanism for three of four detectors.

Information-flow guardrails. (CaMeL [8], FIDES [6], RT-BAS [23]) tag data and enforce statically — provable, but unable to detect attacks whose provenance is correctly labeled while the argument content is malicious.

Output-time guardrails. (NeMo Guardrails, Consecra [19], and the post-hoc monitor Kavach replaces) act after execution — useful for forensics, unable to prevent it.

Drift-detection guardrails. (TaskTracker [1], DeepContext [3], Trajectory Guard [2]) measure trajectory divergence from stated intent across turns. NAVIGATOR sits here at the per-call level, comparing a proposed consequential action against the session’s pinned intent; we report both a completed taxonomy layer and an incomplete authorization mechanism in §6.

Across these categories, the tool-call boundary is the only enforcement point at which the *resolved runtime arguments* of an action are available for inspection before that action executes. Input-time and plan-time guards see only declared intent — the user message or the agent’s reasoning trace — not the concrete arguments the call will run with; output-time guards observe the action only after it has executed, too late to prevent it; and information-flow guards enforce data provenance rather than the semantics of the argument content. This is Kavach’s enforcement point, and it is structurally unreachable by guards in the other categories: they fire at boundaries where the resolved-argument view does not yet, or no longer, exists. §6 reports a consistent, if narrower, coverage gap for the two closest available tool-call-content tools specifically, not a test of the general structural claim above.

Table 1 (§5) places Kavach in context on five axes: enforcement boundary, mechanism, corpus extensibility, verdict combination, and reported AgentDojo / InjecAgent performance.

2.3 Benchmarks and evaluation methodology

The two agent-security benchmarks we evaluate against are:

InjecAgent [21] — 1,054 prompt-injection cases (17 user tools × 62 attacker tools) split into direct-harm (30 tools) and data-stealing (32 tools) settings, each pairing a benign user instruction with a malicious tool-output payload and an expected attacker action. Kavach runs the 544-case data-stealing (DS) subset cold, and the direct-harm subset separately (§4.6).

AgentDojo [9] — 97 user tasks and 629 security test cases across four environments, scored by ASR and utility. We evaluate against its structurally applicable ground-truth attack cases (§4.4) and, at scale, against all 629 real recorded GPT-4o attack trajectories it publishes (§4.5).

A methodological note: reporting attack recall without the accompanying benign false-positive rate conflates detector quality with aggressiveness. We therefore report both jointly for every detection number in §4, following Kapoor et al. [14], rather than gating recall behind a fixed false-positive ceiling.

2.4 OWASP and MITRE taxonomic grounding

Kavach’s attack-pattern corpus is authored against the two most widely cited AI-agent threat taxonomies, not against attacks the team happened to have seen: MITRE ATT&CK/ATLAS (EXECUTOR: T1059, T1546; VAULT: T1552, T1539; CHANNEL: T1041, T1567; NAVIGATOR: T1083 and others) and

the OWASP Top 10 for Agentic Applications 2026, mapped per minister (EXECUTOR: ASI04/05; VAULT: ASI03/09; CHANNEL: ASI07/08; NAVIGATOR: ASI01/10).

3 Kavach Architecture

3.1 Overview

Kavach is a runtime guardrail that interposes between an LLM agent’s tool dispatch and the tool’s actual execution. Every resolved tool call is evaluated by a parliament of four domain-specialized ministers — VAULT (credential and identity abuse), EXECUTOR (dangerous code execution), CHANNEL (data exfiltration and provenance), and NAVIGATOR (unauthorized or drifted agent action), pre-selected per call by a similarity-gated router (falling back to all four when none crosses its threshold). VAULT, EXECUTOR, and CHANNEL are validated detectors with reported detection numbers throughout this paper; NAVIGATOR is the fourth, exploratory component, and we report plainly that it is not yet one (§6) — every aggregate figure including a NAVIGATOR contribution reports it broken out per-minister, not folded in invisibly (e.g. §4.6, §4.7). A deterministic Speaker combines their verdicts by pure veto (any minister’s BLOCK suffices, no score averaging) into BLOCK, ESCALATE, or ALLOW, via a single HTTP endpoint, `POST /hook/parliament`, called synchronously at the host runtime’s pre-execution hook; a verdict other than ALLOW prevents the call from executing. Two further pre-rearchitecture components remain outside the four ministers: TRAJECTORY, a session-level monitor re-confirming or escalating a subset of already-detected cases (disclosed in full, including a real benign false-positive interaction, in §3.7); and COMPASS, a session-intent-drift check that *can* independently upgrade a verdict to BLOCK when active, but whose trigger condition never arises in any of this paper’s dispatch harnesses — confirmed zero occurrences across all 519 + 1,054 + 629 evaluated cases (§4.7).

3.2 Deterministic-primary detection: why, and for which ministers

Kavach’s ministers were originally cosine-primary: each proposed action was embedded and scored against a curated corpus of attack-pattern descriptions, with a per-minister confidence threshold deciding the verdict. Four independent lines of evidence, developed over the course of this project, converged on replacing this design for three of the four ministers.

- **Evidence 1 — a lexical-inflation bug pointed at the fix, not just the bug.** BM25/IDF term weighting over-weighted rare generic tokens (`url`, `user`) because L3 code-syntax text had leaked into the lexical index, false-blocking benign calls (`get_webpage`, `read_inbox`). A regex/exact-match layer on L3 syntax tokens shipped instead of two rejected rerankers (SPLADE, cross-encoder): the false positives were *structural*, and a deterministic match on structure resolved them without touching the semantic layer.

- **Evidence 2 — a targeted dense fine-tune traded one failure mode for a worse one.** A low-rank adapter fine-tuned on the frozen BGE encoder fixed the false positives outright (71%→0%) but regressed real attack recall on 30 genuine InjecAgent attacks from 63.3% to 26.7%: its LOLBIN-shaped training positives sharply separated *that* shape and pulled genuinely different attack shapes (bank transfers, access-grant abuse) away from the decision boundary. The adapter was correctly not merged.
- **Evidence 3 — the corpus gap this exposed was general, not a training artifact.** Adding 62 real business-logic attacks (transfers, access grants) as positive examples revealed they do not match *anything* already in the corpus (mean cosine similarity 0.565, far below any operating threshold) — a structural blind spot for an ordinary-looking business action carrying a malicious argument.
- **Evidence 4 — a proposed fifth minister was formally falsified before being built.** We tested whether cosine similarity could discriminate STEWARD’s target attack class at all, using paired legitimate/malicious AgentDojo calls sharing a tool name and differing only in argument content: similarity deltas were near zero (0.003–0.024). STEWARD was not built — argument-*value* provenance is a different question than action-level semantic similarity, and no corpus expansion closes that gap for a mechanism that only looks at the action as a whole.

The decision. Evidence 1–4 point at one distinction: structural attack signals (a credential-path shape, a network-fetch-to-disk pattern, an argument value’s provenance) are caught more reliably by deterministic parsing than by any refinement of cosine similarity, while genuinely intent-shaped signals may still need a semantic or session-state mechanism. We therefore moved VAULT, EXECUTOR, and CHANNEL to deterministic-primary detection, keeping cosine only as an escalate-only triage layer on the deterministic residual (§3.5). NAVIGATOR remains cosine-primary; a direct deterministic swap was attempted and abandoned at its own go/no-go gate (zero net discrimination gain), and a second attempt at a deterministic authorization layer is reported as a negative result in §6.

3.3 VAULT and EXECUTOR: structural parsing

VAULT and EXECUTOR score each proposed tool call’s arguments with deterministic prefilters (`parliament/prefilters.py`) built on a behavioral-primitive taxonomy: VAULT’s primitives (V1–V6) cover credential paths, dotenv files, and secret environment variables; EXECUTOR’s (E1–E7) cover network-fetch-to-disk, reverse-shell shapes, and persistence writes. Two extraction layers back these, chosen per shell dialect by a latency/correctness trade-off measured directly rather than assumed:

Real AST parsing where it is both correct and fast. For `sh/bash`, arguments are parsed with `bashlex` into a real syntax tree, closing shape-based false negatives a tokenizer cannot (e.g. heredoc content) at ~0.5 ms per call. PowerShell has an equivalently real AST, but invoking it needs a `powershell.exe` subprocess at ~1.4 s per call — 18× over budget — so the real AST is wired in only for `sh/bash`; PowerShell and `command_prompt` remain on the tokenizer below, with the real PowerShell AST kept as an offline/audit tool.

A structural tokenizer for the remaining dialects. For dialects where a real grammar parser is unavailable or too slow, a lightweight shell-quote-aware tokenizer extracts URL-, path-, and credential-shaped tokens directly, with a two-tier credential test (a strong extension/path match, and a broader path-shaped fallback) rather than a single regex.

Full detection numbers, the deterministic/cosine Block/Escalate split, benign false-positive rates, and a held-out-split stability check are reported in §4.3, together with the per-minister benign escalate rates on both populations.

3.4 CHANNEL: session-level taint and value provenance

CHANNEL is inherently a multi-call, session-level detector: the attack shape it targets is a sensitive-data *read* followed by an external *send* in the same session, which no single-call regex can express. It maintains two independent pieces of per-session state.

Taint. A call to a known source tool (a sensitive-data read) marks the session tainted; a subsequent call to a known destination tool is BLOCKED if tainted and the destination is not allow-listed — the allow-list is empty by default, since no corpus evidence supports a “trusted” external recipient distinct from an attacker-controlled one. Source and destination are matched by tool name against curated lists, extended with a tokenizer and a behavior-general fallback checking destination-shaped values (email, IBAN, URL) for any unnamed tool, so a genuinely novel exfiltration tool is not silently invisible.

Provenance. Independent of taint, every destination-tool call’s argument values are classified by where they came from this session: USER_SUPPLIED, SELF (verified identity), OUTPUT_DERIVED (first seen in an earlier tool call’s output this session), or NOVEL (neither). OUTPUT_DERIVED or NOVEL on a consequential call returns ESCALATE, deliberately not BLOCK: paying a bill by reading its own file for the recipient IBAN, or replying to an email just read, are real, legitimate same-session read-then-send chains that must not be treated the same as an attacker-substituted destination (§3.7). Taint’s BLOCK authority previously ignored this classification entirely, hard-blocking legitimate self- or user-named destinations we found and fixed by direct measurement (§4.6); a destination first seen only in a prior tool’s *output* still blocks, deliberately, since that is exactly how an injected destination would appear too.

CHANNEL’s locked reference numbers and full evaluation methodology are reported in §4.4.

3.5 Cosine-ESCALATE-only triage

For VAULT and EXECUTOR, cosine similarity against the original attack-pattern corpus is retained, but only as a triage layer over the deterministic *residual* — cases the structural parsers did not match — and its verdict is hardcoded to ESCALATE: it can never BLOCK. A semantic signal that cannot reliably discriminate (Evidence 4) should not be trusted with an irreversible verdict, but remains useful as a flag for human review on cases a deterministic rule did not anticipate.

An LLM tiebreaker for narrative-phrasing attacks: built, not deployed. VAULT/EXECUTOR’s rules match literal syntax; a prose-described attack (e.g. “append an SSH authorized-keys entry” rather than the literal path) can evade them. We built and validated an escalation tier where a small local model (qwen2.5:7b) only extracts factual flags rather than rendering a verdict directly: 94.1% in pilot, 4/5 on held-out phrasings, one characterized generalization gap (a network-share phrasing the fix does not cover). It is deliberately held back from live dispatch until that gap closes, so no number in §4 includes it — future work, not a contribution.

3.6 Provenance ledger

Every verdict is logged to an append-only, SHA-256 hash-chained SQLite ledger: each row’s hash covers the previous row’s hash plus its own content, so any edit, deletion, or reorder breaks every later hash, detectable in $O(n)$ by /ledger/verify — tamper-evident, not tamper-proof, since write access lets an attacker recompute the whole chain. During this work we found and fixed a case where the verification query itself, not the ledger, omitted a newer column, producing a false tamper report; the fix restored a clean *intact*: true across all rows.

3.7 Design decisions and trade-offs

Deterministic-primary is a per-minister decision, not a blanket policy. NAVIGATOR’s question — was this action authorized by what the user actually asked for — does not reduce to a structural shape the way a credential path or network-fetch-to-disk pattern does, so it was not moved uniformly with the other three (§6). CHANNEL’s provenance mechanism escalates rather than blocks for the same kind of reason: real, common, benign same-session read-then-send patterns in actual trajectory data would be incorrectly stopped by a hard block, so we chose the reversible verdict where that false-positive cost is concrete.

Escalate has a real operational cost, and we report it as such. An Escalate verdict is a request for human review, not a prevented action — the call still reaches the user or operator queue, and someone has to look at it. This has a real cost on legitimate traffic: on the 55-case agent-shaped benign population, 37/55 (67.3%) trigger at least one Escalate somewhere in the pipeline — a majority of ordinary calls, not

a tail case. VAULT and EXECUTOR’s full-pipeline detection numbers (§3.3) report the Block/Escalate split explicitly for exactly this reason: a reader who reads either recall number as “prevented” without checking that split will overstate what the deployed system prevents versus flags for review. A high benign Escalate rate also raises whether an Escalate is informative at all; §4.3 reports Escalate-precision directly rather than leaving that question open.

Synchronous evaluation on the hot path. Kavach evaluates synchronously in the pre-execution hook. Asynchronous evaluation would reduce latency but allow a race between the verdict and execution; we accept the latency cost to maintain the guarantee that no tool executes before a verdict is issued.

A pre-rearchitecture session monitor remains active alongside the four ministers. TRAJECTORY predates this rearchitecture (§3.2) and combines five signals — per-call confidence, session-intent drift, action coherence, denial-echo, cross-minister escalation — into a per-session risk score, sharing its mechanism with the intent-drift detector reported as a negative result in §6. It is not a first-line detector: zero cases counted toward a reported detection number were first identified by TRAJECTORY across every locked evaluation — a minister always fires first. It instead re-confirms or escalates already-detected cases: 148 of the 629 real GPT-4o trajectories in §4.5 (none affecting the reported 467/629), and 3 of the 22 real benign AgentDojo sessions in §4.4 — a hard BLOCK, jointly with NAVIGATOR, on benign traffic our escalate-only CHANNEL mechanism does not produce. We report this plainly: a real false-positive-producing interaction on a live component not subject to the same validation rigor as VAULT, EXECUTOR, and CHANNEL, and not a contribution of this paper. Disclosing this is not resolving it: TRAJECTORY keeps BLOCK authority in every configuration evaluated here, and an operator wanting only the three validated ministers’ verdicts must disable it explicitly.

4 Evaluation

We evaluate Kavach on three independent axes: deterministic-primary detection on a composite evaluation corpus for VAULT and EXECUTOR, assembled because the two standard benchmarks below almost entirely miss them (§4.2–4.3); CHANNEL’s session-level provenance mechanism against real multi-step AgentDojo trajectories (§4.4); and a large-scale validation against AgentDojo’s own recorded model transcripts (§4.5). We report benign false positives jointly with recall throughout, and distinguish in-process from live-HTTP-dispatch measurements, since these diverged in ways that mattered (§4.1).

4.1 An audit discrepancy, and how it was resolved

Early in this work, three measurements of VAULT/EXECUTOR detection rate on the same reference corpus disagreed sharply: 34.2%/34.5% (deterministic layer alone, in-process, since superseded by a VAULT rule fix to the current 35.9%/34.5%,

§3.3), 72.6%/89.9% (full pipeline, in-process), and 14.9%/19.2% (an earlier live-HTTP measurement). Two causes: a duplicate server process corrupted the live-HTTP figure by answering a fraction of requests, and the metric itself differed — one counted BLOCK-or-ESCALATE as a hit, another only the deterministic layer’s contribution. Corrected, the live-HTTP and in-process numbers agreed — a benchmark harness’s own bugs can masquerade as detector performance.

4.2 A composite evaluation corpus for VAULT and EXECUTOR

InjecAgent and AgentDojo, the two standard agent-security benchmarks we rely on elsewhere in this section, almost exclusively stress CHANNEL: InjecAgent’s attack shape is a read-then-send taint chain, and AgentDojo’s is a provenance-mismatched destination value on a consequential call. Neither meaningfully exercises VAULT or EXECUTOR at all — a gap we found live, while validating VAULT/EXECUTOR’s deterministic-primary rearchitecture (§3.2) and discovering we had no real benchmark to validate it against.

Rather than author new synthetic attacks to close this gap, we assembled a unified, composite evaluation corpus for VAULT and EXECUTOR from already-real, externally-validated attack data (AtomicRedTeam, GTFOBins, BFCL) under one runner, one wire format, and one scoring convention. This is deliberate: a self-authored attack set risks unconsciously being shaped to be catchable by the very system that writes it, and reusing independently-authored corpora avoids this risk structurally rather than by discipline alone.

The corpus’s other 207 cases, and Kavach’s rate on all 519. Target sets overlap, so per-minister counts do not sum to 519: VAULT (117) and EXECUTOR (203) share 8 (312 distinct), leaving 207 targeting neither. Within those 207 (all AtomicRedTeam), CHANNEL (72) and NAVIGATOR (144) share 9 (union 207); their *primary* evaluation remains InjecAgent (§4.6) and AgentDojo (§4.4, §4.5). Any-minister detection on these 207 is 206/207 (99.5%), though each minister’s own verdict alone catches only 23/72 (31.9%, CHANNEL) and 92/144 (63.9%, NAVIGATOR) — the rest caught by cosine-triage or a different minister. Kavach’s overall any-minister rate across all 519 — the same denominator the baselines below use — is 515/519 (99.2%), a looser metric than the strict per-target recall reported next, not a like-for-like comparison. NAVIGATOR’s own contribution, per §3’s disclosure convention: sole contributor on 9/519 (1.7%), so 506/519 (97.5%) survive with it removed.

Reuse and scoring. Attack cases are drawn from AtomicRedTeam and GTFOBins; benign calls from BFCL (793, general-purpose) plus a 55-case agent-shaped benign set, reported separately rather than pooled. All cases are dispatched through the same `/hook/parliament` path under one per-minister scoring convention and one benign false-positive convention, purely to make an existing evaluation gap measurable, not as a standalone artifact. Each attack case carries a `target.patterns` list naming which minister(s)

it should trigger; we report both an overall detected rate and a per-minister hit rate, since a case can be detected overall while its targeted minister misses. On the benign side, BLOCK is a hard false positive and ESCALATE is reported separately, never conflated with BLOCK or a clean ALLOW — the same convention CHANNEL’s provenance mechanism uses (§3.4).

4.3 Results: VAULT and EXECUTOR

On this 519-case AtomicRedTeam/GTFOBins reference corpus (117 VAULT-targeted, 203 EXECUTOR-targeted; `target.patterns` is unmodified since its original authoring commit, confirmed via `git diff`, so it is independent ground truth), the deterministic layer alone recovers VAULT 35.9% (42/117) and EXECUTOR 34.5% (70/203); adding the cosine-escalate-only triage layer (§3.5) raises full-pipeline detection to VAULT 74.4% (87/117) and EXECUTOR 90.1% (183/203) — of which 42 points (VAULT) and 70 points (EXECUTOR) are hard-BLOCK from the deterministic layer, and 45 and 113 points respectively are ESCALATE-only from cosine triage — with zero new hard-block false positives on either benign population (793 BFCL, 55 agent-shaped). Benign ESCALATE rates on the two populations (agent-shaped/BFCL) are: VAULT 5.5%/13.4%, EXECUTOR 0%/14.9%, CHANNEL 40.0%/4.5%, NAVIGATOR 21.8%/10.8%. A held-out even/odd split confirms the full-pipeline rate is stable, not an artifact of partition: VAULT 71.2% (42/59) versus 77.6% (45/58); EXECUTOR 89.2% (91/102) versus 91.1% (92/101) — both halves reported explicitly rather than only their average.

Escalate-precision, not just recall. A high benign Escalate rate could make an attack-side Escalate uninformative, so we report Escalate-precision — of a minister’s own Escalate verdicts, what fraction land on a true attack — per benign population rather than pooled: VAULT is 93.8% (45 true vs. 3 false) on agent-shaped benign but 29.8% (45 vs. 106) on the larger, more general BFCL population; EXECUTOR is 100% (113 vs. 0) agent-shaped but 48.9% (113 vs. 118) on BFCL. Precision is high on the population closer to Kavach’s deployment context and lower on generic traffic, an effect of BFCL’s volume amplifying a modest per-call rate rather than a change in the mechanism; both populations’ benign:attack ratios are corpus-constructed, not a measured production base rate, so these figures should be read relative to each other, not as an absolute expectation.

The controlled comparison is architecture vs. architecture, full pipeline vs. full pipeline, not component vs. system. Pre-rearchitecture cosine-primary — same harness, same 519 cases, code unmodified, Block-or-Escalate a hit on both sides (re-verified against the current corpus: 79/117, 174/203, confirming the convention rather than asserting it) — recovers VAULT 67.5% (79/117) and EXECUTOR 85.7% (174/203), at 4.9% (39/793) hard-block FPR on general-purpose benign (BFCL). The 74.4%/90.1% pipeline above (not the deterministic layer’s own 35.9%/34.5%, a decomposition of it) wins both axes: a modest, mechanism-attributable recall gain (+6.9pp VAULT, +8 cases; +4.4pp EXECUTOR, +9 cases) alongside

the decisive result, hard-block FPR eliminated ($4.9\% \rightarrow 0\%$) — a same-team baseline showing the rearchitecture improves on the design it replaced on this same 519-case corpus.

We additionally compare against two independent, non-Kavach baselines on the identical 519-case corpus and both benign populations, matching their denominator against Kavach’s own 515/519 (99.2%, §4.2) rather than the narrower per-target figures above. A naive keyword/regex blocklist (~ 30 hand-written patterns, no corpus, no tuning) recovers 12.1% (63/519) at 0% FPR — the expected floor for an uncorpus-driven list. A zero-shot LLM judge (local `qwen2.5:7b`, no corpus, single fixed prompt, resolved call only) recovers 78.8% (409/519) — below Kavach’s Block-or-Escalate 99.2% but above its single-tier, hard-BLOCK-only 29.5% (not on equal footing; see below) — at a false-positive cost Kavach’s deterministic layer avoids entirely: 1.4% BFCL, 14.5% agent-shaped. A larger model (`qwen2.5:32b`, same prompt and populations) does not simply dominate: recall falls to 71.3% (370/519) while FPR drops to 0.4% combined (0% agent-shaped) — a different point on the same recall/precision trade-off, not a strictly better one, indicating the trade-off is inherent to LLM-judge-style detection rather than an artifact of model scale. Read together with the cosine-primary ablation above: a naive filter under-detects, a generic LLM judge (at either scale) trades recall against precision but cannot match both, and cosine-primary approaches neither Kavach’s recall nor its zero-FPR guarantee.

This comparison is not symmetric; we disclose the gap. The three baselines above are single-tier (flagged or not); Kavach’s figures mix Block and the softer Escalate. As a single hard-block action, Kavach’s own recall on these 519 cases is 153/519 (29.5%) — below both judges’ flagged rate — and its own any-minister FPR on the same benign populations is 29.6%/67.3% (BFCL/agent-shaped), *above* either judge’s (1.4%/14.5% at 7B, 0.4%/0% at 32B). Weigh 29.5%/29.6%/67.3% against these baselines’ binary figures, not 99.2%/0%; the latter pair is a fair comparison only against another Block-or-Escalate system, such as the cosine-primary ablation above.

This split addresses partition luck, not rule-authoring circularity: the deterministic rules were authored by inspecting misses across the full corpus, so both halves were visible during rule development. The generalization audit (§6.1) attempts to measure past this limit, and has its own limit in turn: its scenarios are author-written, testing vocabulary novelty rather than an independently sourced distribution.

4.4 CHANNEL: session-level provenance

CHANNEL’s provenance mechanism (§3.4) is evaluated on two real populations: 22 real multi-step benign sessions and the AgentDojo attack cases whose two-call read-then-send shape is structurally applicable to CHANNEL’s mechanism. On the benign population, 5/22 sessions correctly escalate via CHANNEL with 0 hard-block false positives from CHANNEL itself — exactly the sessions with a legitimate destination

value (an IBAN, most commonly) read earlier in the same session rather than typed by the user (§3.4). A further 3/22 are hard-blocked by an unrelated TRAJECTORY/NAVIGATOR interaction, never blended into the 5/22 figure and reported in full in §3.7. On the attack side, 5/5 structurally applicable AgentDojo cases correctly escalate, verified twice: against the benchmark’s minimal `ground_truth()` sequence, and independently against AgentDojo’s own full recorded model trajectories for the same cases — real, longer sessions with benign calls interleaved. This second check caught a bug in our own verification script (a benign call after the attack step could overwrite an earlier correct escalation) before we reported the number as final; the corrected script confirms genuine 5/5.

4.5 Large-scale real-trajectory validation

The measurements above use either a synthetic reference corpus or a small, hand-verified set of AgentDojo cases. To validate at scale on real model behavior, we replayed all 629 real recorded attack trajectories for GPT-4o (`gpt-4o-2024-05-13`) across AgentDojo’s four suites’ `important.instructions` family, sourced directly from AgentDojo’s own published `runs/` data — real transcripts with real tool calls and outputs, not a minimal ground-truth replay.

Headline result. Kavach fires (BLOCK or ESCALATE, any minister, at any point in the session) on 467/629 (74.2%) of these real trajectories: banking 119/144 (82.6%), slack 105/105 (100%), travel 63/140 (45.0%), workspace 180/240 (75.0%).

Benign control, same criterion. 74.2% had no benign comparison at this granularity, so we replayed AgentDojo’s 97 recorded no-injection GPT-4o trajectories under the identical criterion. Kavach fires on 64/97 (66.0%), but only 4/97 (4.1%) are hard-BLOCK (the rest Escalate); per-minister credit: CHANNEL 28, EXECUTOR 19, NAVIGATOR 12, VAULT 5 — NAVIGATOR is a minority contributor. On the attack side, hard-Block alone fires on 7/629 (1.1%), *lower* than the benign 4.1%, since these are session-level provenance attacks CHANNEL deliberately escalates rather than hard-blocks (§3.7); hard-Block is reserved for VAULT/EXECUTOR’s structural certainty, rarely triggered by multi-step sessions. This recontextualizes rather than invalidates 74.2%, consistent with §3.7’s 67.3% Escalate rate.

Diagnosing the apparent misses. Of the 162 trajectories on which Kavach did not fire, we traced every case to its actual tool-call content rather than reporting a bare miss rate. AgentDojo’s own `security` field marks a trajectory as a successful attack whenever the environment state changed at all relative to the pre-trajectory state and does not exactly match the injected goal — conflating “the injected attack succeeded” with “the model made no consequential call related to the injection at all.” 131 of the 162 misses fall in the latter category, verified by extracting every tool call from each miss’s transcript against a fixed consequential-tool list: the model never invoked any consequential tool in connection

with the injected instruction, so there was no unauthorized action for any minister to catch, regardless of what AgentDojo’s field reports. A further 12 are trajectories where AgentDojo’s own field already indicates the injection did *not* succeed (8 `workspace`, 4 `travel`) — not Kavach misses either, since there is no successful attack to have caught. The remaining 19 are genuine misses.

The remaining 19 real misses, categorized by mechanism. We categorize each by whether the underlying attack has a destination *value* to trace (CHANNEL’s domain) or is purely an unauthorized *action* with no data flow at all (closer to NAVIGATOR’s domain, §6). Five cases (`travel`, all `reserve_hotel`) name an attacker-substituted external party — a real destination value, of the same shape as `send_money`’s recipient IBAN — but the tool was never added to CHANNEL’s destination-tool list, a scope gap rather than a mechanism failure. Fourteen cases (`banking`: 11; `workspace`: 3, `delete_file` and `append_to_file`) mutate existing account or same-account state with no new destination value introduced at all — no value for any provenance mechanism to trace, since these attacks ask “should this action have happened,” not “where did this value come from.” Adding the tool to CHANNEL’s destination list to close the five `reserve_hotel` cases introduces a new false positive instead: the value extractor recognizes only email/IBAN/phone/URL shapes, not proper-noun business names. We reverted this change and report the gap as a documented scope limitation (§6) rather than a closed fix.

Scope of this validation. This covers one model (`gpt-4o-2024-05-13`) and one attack family, the complete recorded trajectories AgentDojo publishes; transfer to other models’ tool-calling styles or subtler injection framings is untested, a deliberate trade of breadth for realism.

4.6 InjecAgent

InjecAgent’s full corpus is 1,054 cases: 510 direct-harm (DH) and 544 data-stealing (DS), each the cross product of InjecAgent’s 17 user tasks with its 30 DH / 32 DS attacker instructions. Both are dispatched as a two-step sequence — a source read, then the destination send (DS) or the direct-harm action itself (DH) — so CHANNEL’s session-scoped taint/provenance tracker can observe the read-then-act pattern in both settings; DH’s dispatch was corrected to this two-step form as a methodology fix, not a change to any minister’s detection logic.

On DS, Kavach achieves **544/544 (100%) strict (hard-Block) recall**, entirely via CHANNEL: every one of the 544 cases is decided by CHANNEL’s taint mechanism (`CHANNEL-TAINT :read-then-send`), with VAULT and EXECUTOR returning ALLOW on all 544 — this result exercises CHANNEL’s multi-step provenance/taint mechanism directly, not VAULT/EXECUTOR’s deterministic layer.

The paired 0% benign hard-block FPR did not test this mechanism, and we found and fixed a real gap. InjecAgent’s benign dispatch is single-call — no destination step follows the

source-read — so taint’s precondition is never presented to it; the 0% figure is a structural guarantee of the dispatch, not a measurement. Direct test: 32 genuine two-step benign actions built from InjecAgent’s own tool vocabulary, dispatched live. Legitimate self-forwarding was hard-blocked 32/32 times, since taint’s only carve-out was a literal account-identity match this harness never supplies. We fixed the two positively confirmable cases — destination matches the session’s own identity, or was named in the user’s own instruction — by having taint defer to CHANNEL’s own provenance classification (§3.4); re-validated with zero change to DS’s 544/544 recall or the 519-case reference corpus (checkpoint gate unchanged: 87/117 VAULT, 183/203 EXECUTOR). The harder case — a destination first seen only in an earlier tool’s output, which could be a legitimate invoice recipient or exactly how an injected destination is smuggled in — remains an open gap, deliberately left blocking rather than resolved by this fix, since resolving it correctly needs the still-incomplete NAVIGATOR authorization work (§6).

DH tells a different, weaker story: no individual minister reaches hard BLOCK on a DH case. 221/510 (43.3%) reach the loose (block-or-escalate) level; *decided-by* counts (the one minister the Speaker credits per case, mutually exclusive by construction, vs. counting any minister that fired, which gives EXECUTOR 34 and NAVIGATOR 51 since 51 cases have two-plus ministers firing together) split this as CHANNEL (153), VAULT (34), EXECUTOR (17), and NAVIGATOR (17); the remaining 289/510 (56.7%) are missed entirely. CHANNEL’s provenance mechanism escalates 153/510 (30%) via a novel-value classification, while its own hard-block contribution stays exactly 0%, as designed. Separately, 16/510 (3.1%) reach a hard BLOCK in aggregate via TRAJECTORY (§3.7), not any minister — consistent with its already-disclosed role, not a change to any minister’s detection contract. DH remains a genuine, unresolved gap, not a strength of the current architecture, and is not folded into DS’s headline number. This snapshot predates CHANNEL’s destination-shape generalization fix (Appendix A); re-measuring after it lands is future work, kept out here so §4 reflects one consistent snapshot, not a pre/post-fix mix.

On the paired benign side (1,054 user-instruction prompts, one per attack case), hard-block FPR is **0%** — no benign prompt is ever hard-blocked — but the soft-escalate rate is **35.2% (371/1,054)** by the same decided-by convention, again led by CHANNEL (248), with EXECUTOR (62) and NAVIGATOR (61) contributing the rest and VAULT at 0. DS’s strong recall and DH’s weak recall should both be read against this FPR context, not in isolation; across the combined 1,054-case corpus, Kavach’s strict (hard-BLOCK-only) recall is 544/1054 (51.6%) — entirely DS, since DH’s strict contribution is 0% — the number missing from a strict, apples-to-apples row in Table 1.

4.7 Cost

Kavach’s latency is bimodal, and we report the split rather than a blended figure: 17 of 159 timed calls complete under

90 ms (p50 46 ms, p95 86 ms), the remaining 142 in over 680 ms (p50 733 ms, p95 1437 ms), with no call in between. The split does not cleanly track whether a deterministic rule short-circuited the call — 26 of 159 were short-circuit-eligible and matched, of which only 17 returned fast despite an identical decision path — traced to the pipeline’s ledger-completeness design: every response, short-circuited or not, spawns a background task running the skipped COMPASS (§3) and NAVIGATOR scoring for audit purposes, competing for the same CPU-bound embedding inference as the next request’s foreground call under sequential dispatch. This is a scheduling artifact of our single-process CPU measurement, not the detection logic; whether bounding or deferring it in a real deployment actually closes the gap is unmeasured (§6). On GPU-equipped hardware we have separately observed the embedding path at ~78 ms p50 (RTX 4090, steady state); verdicts are hardware-independent (replaying 107 reference cases with the encoder pinned to CPU produced zero verdict or fired-minister differences against the GPU baseline). The bashlex parse for `sh/bash` (§3.3) measures ~0.5 ms per call, negligible on either path.

5 Related Work

Positioning relative to concurrent runtime work. MXC (Microsoft Execution Containers, Build 2026) and ClawGuard [22] both harden the OpenClaw runtime Kavach was originally built against. MXC is an OS-level policy sandbox — orthogonal to Kavach’s argument-content question, since an authorized agent can still be manipulated into exfiltrating data within its own policy. ClawGuard’s 0% AgentDojo attack-success rate is under hand-authored rules only, its rule-derivation component unevaluated. DeepContext [3] and PSG-Agent [20] are architecturally adjacent (Table 1); our differentiation is that mechanism choice (deterministic vs. cosine) is made per detection domain on direct evidence, with generalization reported explicitly (§6) rather than only success.

Plan-time authorization systems. PlanGuard [11], IntentGuard [13], and Progent [18] are the closest precedents for NAVIGATOR’s authorization question, each at the plan level rather than the tool-call level. PlanGuard’s hard-constraint stage plus LLM verifier reports 100% attack-blocking on InjecAgent; NAVIGATOR’s analogous pinned-intent design is instead an incomplete, three-attempt negative result (§6). IntentGuard’s user-confirmation-on-alert is close in spirit to CHANNEL’s escalate-not-block verdict (§3.4). Progent’s progressive allowlist reports an AgentDojo ASR reduction from 39.9% to 1.0%, an upper reference point, not a number Kavach reproduces.

Red-team and scanning tools. garak [10], PyRIT [16], and LLM-Guard [17] probe or scan at the prompt/input boundary pre-deployment rather than enforcing at the tool-call boundary at runtime — complementary to, not competing with, Kavach.

Table 1 (Appendix B) situates Kavach against 13 systems on five axes, drawn from each cited paper’s own evaluation setup — benchmark subset, environment, and threat model all vary — so it is a map of enforcement boundaries and mechanisms, not a leaderboard; no number in it is directly comparable across rows. Full comparison in Appendix B.

6 Limitations

Latency is a real, unresolved cost of the no-GPU deployment this paper evaluates. 142 of 159 timed calls (89%) take over 680 ms (p95 1.44 s) on the synchronous hot path (§4.7) — the exact configuration validated throughout §4. Whether deferring the background COMPASS/NAVIGATOR audit work closes this gap is asserted, not measured; a real deferred-execution deployment is future work.

No head-to-head with ClawGuard specifically. No controlled head-to-head against ClawGuard’s implementation (§5) exists: it intercepts live sessions with no dataset-replay harness compatible with our pipeline.

No AgentDojo ASR with Kavach as the live defense. Table 1 reports per-mechanism trajectory analysis (§4.5) rather than a single AgentDojo ASR, the field’s standard metric. Producing it needs AgentDojo’s own harness with Kavach wired in as its live defense (`benchmarks/run_agentdojo_kavach.py` is built for this), driving a live model through all 97 user tasks under attack, not a replay: not completed before submission, the most direct remaining gap here.

External tool-call-content baselines are structurally inapplicable, not weaker. We attempted comparison against the two closest external tools; both proved structurally inapplicable, not merely weaker: CodeShield’s supported languages (C/C++/C#/Java/JavaScript/PHP/Python/Rust) exclude the shell, PowerShell, and cmd dialects comprising ~87% of our corpus, and gitleaks — a secret-value scanner — finds zero hits on all 519 cases, since they encode credential-access techniques rather than embedded secrets: corroborating the enforcement-boundary gap targeted here, not a performance comparison.

Tamper-evident, not tamper-proof. The hash-chained ledger detects post-hoc edits to any recorded decision, but an attacker with database write access can recompute the entire chain; defeating that needs an external append-only anchor, not implemented here. We found and fixed a real bug in the ledger’s own *verification* query during this work (§3.6).

Curated corpus and deterministic rule set. Both the cosine-triage corpus and the deterministic prefilters’ rule sets are authored against published taxonomies (MITRE ATT&CK/ATLAS, AtomicRedTeam, GTF0Bins) and known attack corpora (InjecAgent, AgentDojo); a genuinely novel technique outside both may go uncaught (§6.1).

The reserve_hotel gap is representative of a broader, unfixed class. §4.5 categorizes the 19 genuine real-trajectory misses in full: five (`reserve_hotel`) are a scoped CHANNEL

destination-tool gap whose natural fix we tried and reverted (new false-positive class); the remaining fourteen have no destination value to trace at all, structurally not closeable by any refinement of CHANNEL’s mechanism — the shape of question §6 reports as unresolved.

Single embedding model. Cosine-triage and NAVIGATOR use one encoder (**bge-base-en-v1.5**), chosen for offline reproducibility, not comparative ablation; whether another retrieval encoder shifts the residual detection rate is left to future work.

PowerShell’s real AST is correct but too slow for the live path. §3.3 reports why PowerShell calls are scored by the structural tokenizer rather than a real AST live; a faster in-process PowerShell AST is a scoped next step we did not pursue this cycle.

NAVIGATOR authorization layer: a hard-constraint layer still in progress, after three failed heuristics. NAVIGATOR is the one minister whose deterministic rearchitecture is not yet complete. A direct deterministic swap mirroring VAULT/EXECUTOR/CHANNEL showed zero net discrimination gain and was abandoned; a second attempt reframes the question as authorization rather than similarity (a pinned, immutable session intent, classifying each tool as observational or consequential, then a hard-constraint layer deciding whether a consequential action is *derivable* from that intent). The first two pieces are complete and validated, including under the audit below. The hard-constraint layer failed three independently designed heuristics against a frozen discriminability criterion: (1) tool-name/instruction-text overlap, evaded by attacker text sharing surface words with the legitimate request; (2) argument-value overlap alone, wrongly flagging legitimate values read earlier in the session; (3) a provenance-aware version of (2), still over the frozen threshold.

A proposed alternative for NAVIGATOR: intent-drift, explored and not validated. Independently of the hard-constraint layer above, we explored a second candidate for NAVIGATOR — flagging a call as drift when its embedding similarity to the session’s original user-stated goal falls below a threshold — which reached 95.2% recall at 0% FPR on an initial 100-case dataset (83 attack, 17 benign), but did not generalize: enlarging the benign sample to 39 real multi-step AgentDojo sessions raised FPR to 35.9% at the same threshold, since ordinary mid-session follow-ups (e.g., checking a balance right after a payment) are semantically distant from the session’s *original* goal despite being legitimate, and held-out recall on the enlarged set collapsed to 22–56%, consistent with the broader drift-detection literature’s own reported difficulty generalizing past its original benign distribution (§2). We report this as future work, not a contribution: NAVIGATOR’s deployed detector remains the cosine-corpus mechanism in §6.1, unchanged.

No adaptive-attacker evaluation, and a first-pass obfuscation check. Every attack corpus here is static: no attacker

knows Kavach’s rules or adapts to them. Deterministic matching is historically evadable — encoding, string construction, variable indirection — and while bashlex’s AST closes a class of these for sh/bash, PowerShell and cmd remain on a tokenizer. First-pass check, sh/bash: 185 previously-detected cases through base64 encode-and-pipe (applies to all 185); split and indirection require a literal anchor span present in only 75, so those two are scored on that subset. True per-mechanism evasion — after correctly distinguishing genuine evasion from TRAJECTORY’s session-level re-confirmation merely relabeling an already-correct verdict (§3.7) — is 7.0% (encode, 13/185), 4.0% (split, 3/75), 25.3% (indirect, 19/75): 35/335 (10.4%) evaded overall, every one caught by a different mechanism, so none reached the agent unblocked. One backup is itself imprecise: encode’s backstop is a single NAVIGATOR pattern matching the literal `echo BASE64 | base64 -d | sh` shape, confirmed to also block an equivalent benign command — a blanket rule, not intent-aware detection. Extended to PowerShell/cmd on the same convention (263 detected cases; encode applies to all 263, split/indirect to the 73 with an anchor), dialect-native transforms — `-EncodedCommand`, caret-escaping, split/indirection: true evasion is markedly higher, 28.9% (encode, 76/263), 16.4% (split, 12/73), 19.2% (indirect, 14/73), 102/409 (24.9%) overall — confirming the tokenizer-not-AST hypothesis empirically. Unlike sh/bash, not every case is backstopped here: 97/102 are still caught by a different mechanism, but 5 (all `encode`) reach ALLOW entirely — a genuine, unmitigated miss, not merely a per-mechanism one. Still a first pass, not adversarial; an attacker targeting two or more mechanisms at once is untested, and §4’s rates should be read as non-adaptive until then.

6.1 Generalization beyond the benchmark vocabulary

Kavach’s deterministic rules were, in large part, authored by mining detection misses on the exact benchmarks evaluated against (§4.2), a real risk of circular validation. We therefore ran an independent adversarial audit before submission: a fictional agent environment (“AtriumOS”) with an entirely invented tool vocabulary sharing no name, term, or phrasing with InjecAgent, AgentDojo, or our corpus. Verdict: EXECUTOR generalizes cleanly, VAULT mostly does, CHANNEL’s escalation path generalizes but its hard-block path does not, and NAVIGATOR’s cosine mechanism does not transfer. A second, independent check against AgentHarm’s (UK AISI, ICLR’25) published tools corroborates this: 81.8% in-scope (36/44, mostly ESCALATE), consistent with §3.7’s disclosure. Full methodology in Appendix A.

7 Conclusion

Kavach demonstrates that a deterministic-primary architecture outperforms the cosine-primary design it replaced when the attack signal is structural, with its negative results as much a contribution as its numbers.

A Generalization Audit Methodology and Full Results

Methodology. §6.1 summarizes why this audit exists and its top-line verdict; this appendix gives the per-minister mechanism and numbers. AtriumOS’s invented vocabulary (`FacilityConfigRead`, `HvacDiagnostic`, and so on) was dispatched as 18 attack and 18 matched-benign scenarios, one per minister’s detection shape, tested before AtriumOS existed, as a case for *transfer* rather than a headline claim. Not evenly split across ministers (8 EXECUTOR, 7 VAULT, 3 combined CHANNEL/NAVIGATOR); the latter two rest on very few cases and should be read as illustrative, not statistically powered.

- **EXECUTOR** — **generalizes cleanly.** Caught all eight novel-vocabulary execution attacks via structural shape matching, independent of tool identity. One structural false positive (a network-fetch-to-disk predicate on a benign call) was found and fixed pre-submission; parser test suites remain green (27/27, 37/37).
- **VAULT** — **mostly generalizes, one inherent limit.** Caught 5/7 novel-vocabulary credential attacks via structural credential-path and secret-format matching. One miss is a fixable surface-form gap; the other is architectural: a named-service secret-key rule is a literal allowlist by construction, so a genuinely novel secret-variable name cannot match it.
- **CHANNEL** — **split, since partially closed.** Its provenance/escalation path generalizes: a behavior-general destination-value fallback correctly escalated a novel send-tool’s attacker address. Its hard-block path originally did not (destination status came from a benchmark-specific tool-name dictionary alone); we wired the same fallback in, closing the gap for shape-matchable destinations (verified: a novel IBAN-shaped value). A destination that is *not* shape-matchable (e.g. `reserve_hotel` elsewhere in this paper) remains invisible to it.
- **NAVIGATOR** — **does not currently generalize as a detector.** Cosine similarity separates novel attack from benign by only 0.045 (0.590 vs. 0.545 mean), and the BM25 lexical gate makes this worse off-benchmark, collapsing both to ~ 0.38 — below the escalation floor. Structural, not incidental: the mechanism giving NAVIGATOR its low on-benchmark FPR removes its off-benchmark signal. Its one general component — an observational/consequential classifier — labeled all four novel tools correctly, but is a *taxonomy, not a detector*: it does not decide whether an action was *authorized*, needing the pinned-intent layer reported incomplete in §6.

A second, genuinely independent check: AgentHarm. AtriumOS is author-written; to test transfer against data this team did not author at all, we used AgentHarm’s (UK AI Safety

Institute, ICLR 2025) published synthetic tool implementations — 80 harmful-scenario and 80 benign-scenario Python tool files, MIT-licensed, fetched directly rather than through AgentHarm’s official gated benchmark (Inspect framework, API keys, and not tool-call structured regardless). A local `qwen2.5:7b` generated one realistic call per tool from its docstring (a first heuristic attempt was discarded as unfair to both sides; 1 of 80 harmful tools failed argument generation, leaving 79). Every one of those 79 cases was hand-classified, once, before computing any number, as in-scope or out-of-scope for Kavach’s four-domain threat model. Of 44 in-scope cases: 36 caught (81.8%; 5 hard-BLOCK, 31 ESCALATE), 8 missed — anti-forensics, public exploit-bragging, phishing-asset generation, a phishing email with a malware attachment, a tool-name vocabulary gap (`transfer_money_crypto`, the same class as CHANNEL’s gap above), two malware-named file uploads, and a no-argument credential harvest matching an earlier finding. The 80 benign cases: 0% hard-block, 56.3% Escalate, consistent with §3.7’s disclosed rate.

B Extended Related Work Comparison

Table 1 extends §5’s summary to all 13 systems given their own row, grouped by enforcement boundary to match §2’s taxonomy. Caveats on comparability are in the caption.

Table 1: Comparison of agent guardrails on five axes: enforcement boundary, primary mechanism, whether the corpus/rule set is extensible without retraining, verdict combination, and best-reported AgentDojo/InjecAgent numbers (n/r = not reported; ‡ = source paper’s own number, not independently reproduced here). Lower ASR / higher Recall is better; not a leaderboard, since benchmark subset and threat model vary system to system.

System	Boundary	Mechanism	Ext.?	Verdict	AgentDojo ASR ↓	InjecAgent Recall ↑
PromptGuard 2	input	DeBERTa classifier	no	threshold	7.5%	n/r
PlanGuard [11]	plan	isolated planner + verifier	no	hard-constraint	n/r	100%‡
IntentGuard [13]	plan	intent trace + origin	no	alert-first	n/r	n/r
Progent [18]	tool-call	progressive allowlist	yes (policy)	policy-match	1.0%‡	n/r
CodeShield	output	Semgrep+regex	yes (rules)	threshold	n/r	n/r
ShieldAgent	tool-call	rule circuits	yes (docs)	probabilistic	n/r	90.1%
PSG-Agent	tool-call	4-component	partial	cross-turn accum.	n/r	n/r
Kavach (ours)	tool-call	determin.+embed.	yes (rules+corpus)	veto	n/a‡	51.6% strict (544/1054)‡
CaMeL	IFC	dual-LLM + taint	no	provable	23% (sec)	n/r
RTBAS	IFC	saliency screener	no	threshold	2% loss only	n/r
DeepContext	session	RNN over embeds	no	binary	n/r	n/r
Trajectory Guard	trajectory	Siamese RAE	no	score	n/r	n/r
garak [10]	input (test)	vuln. probes	yes (probes)	per-probe	n/r	n/r
PyRIT [16]	input (test)	red-team orch.	yes (attacks)	n/a	n/r	n/r

Veto (Kavach): any minister returning BLOCK vetoes the call unconditionally, no score averaging. **Extensibility** (Kavach): deterministic prefilters extend by adding a structural rule; the cosine-triage corpus by adding a natural-language attack-pattern description, re-embedded without retraining. **‡**Strict (hard-BLOCK-only) recall on the full 1,054-case corpus, entirely from the 544/544 data-stealing subset; direct-harm’s strict contribution is 0%, a genuine gap (§4.6). **§**Kavach reports per-mechanism trajectory analysis (§4.5) rather than a single AgentDojo ASR.

References

- [1] Sahar Abdelnabi, Aideen Fay, Giovanni Cherubin, Ahmed Salem, Mario Fritz, and Andrew Paverd. 2024. Are you still on track!? Catching LLM Task Drift with Activations. *arXiv:2406.00799* [cs.CR]
- [2] Laksh Advani. 2026. Trajectory Guard – A Lightweight, Sequence-Aware Model for Real-Time Anomaly Detection in Agentic AI. AAAI 2026 Trustworthy Agentic AI Workshop.. *arXiv:2601.00516* [cs.LG]
- [3] Justin Albrethsen, Yash Datta, Kunal Kumar, and Sharath Rajasekar. 2026. DeepContext: Stateful Real-Time Detection of Multi-Turn Adversarial Intent Drift in LLMs. *arXiv:2602.16935* [cs.AI]
- [4] Zhaorun Chen, Mintong Kang, and Bo Li. 2025. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. *arXiv:2503.22738* [cs.CR]
- [5] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, et al. 2025. LlamaFirewall: An Open Source Guardrail System for Building Secure AI Agents. *arXiv preprint arXiv:2505.03574* (2025).
- [6] Manuel Costa, Boris Köpf, Aashish Kolluri, et al. 2026. Securing AI Agents with Information-Flow Control. In *Proceedings of the 2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. *arXiv:2505.23643*
- [7] CrowdSec. 2026. CVE-2025-34291 Exploited in the Wild: LangFlow AI Framework Under Fire. <https://www.crowdsec.net/vulntracking-report/cve-2025-34291>.
- [8] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, et al. 2026. Defeating Prompt Injections by Design. In *Proceedings of the 2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. *arXiv:2503.18813* [cs.CR]
- [9] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*. *arXiv:2406.13352* [cs.CR]
- [10] Leon Derczynski, Erick Galinkin, Jeffrey Martin, Subho Majumdar, and Nanna Inie. 2024. garak: A Framework for Security Probing Large Language Models. *arXiv:2406.11036* [cs.CL]
- [11] Guangyu Gong and Zizhuang Deng. 2026. PlanGuard: Defending Agents against Indirect Prompt Injection via Planning-based Consistency Verification. *arXiv preprint arXiv:2604.10134* (2026).
- [12] Feiran Jia, Tong Wu, Xin Qin, and Anna Squicciarini. 2025. The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents. *arXiv:2412.16682* [cs.CR]
- [13] Mintong Kang, Chong Xiang, Sanjay Kariyappa, Chaowei Xiao, Bo Li, and Edward Suh. 2025. Mitigating Indirect Prompt Injection via Instruction-Following Intent Analysis. *arXiv preprint arXiv:2512.00966* (2025).
- [14] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2024. AI Agents That Matter. *arXiv:2407.01502* [cs.LG]
- [15] Hao Li, Xiaogeng Liu, Hung-Chun Chiu, et al. 2025. DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents. In *Advances in Neural Information Processing Systems*. *arXiv:2506.12104*
- [16] Microsoft AI Red Team. 2024. PyRIT: Python Risk Identification Tool for generative AI. <https://github.com/Azure/PyRIT>.
- [17] Protect AI. 2024. LLM Guard: a security toolkit for LLM interactions. <https://github.com/protectai/llm-guard>.
- [18] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI Agents with Privilege Control. *arXiv preprint arXiv:2504.11703* (2025).
- [19] Lillian Tsai and Eugene Bagdasarian. 2025. Contextual Agent Security: A Policy for Every Purpose. *arXiv:2501.17070* [cs.CR]
- [20] Yaozu Wu, Jizhou Guo, Dongyuan Li, et al. 2025. PSG-Agent: Personality-Aware Safety Guardrail for LLM-based Agents. *arXiv:2509.23614* [cs.CR]
- [21] Qiushi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. 10471–10506. *arXiv:2403.02691* [cs.CL]
- [22] Wei Zhao, Zhe Li, Peixin Zhang, and Jun Sun. 2026. ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents Against Indirect Prompt Injection. *arXiv preprint arXiv:2604.11790* (2026).
- [23] Peter Yong Zhong, Siyuan Chen, and Others. 2025. RTBAS: Defending LLM Agents Against Prompt Injection and Privacy Leakage. *arXiv:2502.08966* [cs.CR]

Use of Generative AI

Generative AI tools (Anthropic’s Claude, via Claude Code) were used throughout this project’s development: implementing detection mechanisms from the authors’ designs, running benchmark evaluations, drafting text from the authors’ findings, and assisting with code review. All design and interpretation were directed and verified by the authors.